



NZ E-COMMERCE & DELIVERY TRACKING PIPELINE

Jagdeep Singh

Contents

1. Introduction	2
2. Aim & Purpose.....	2
3. Real-World Business Problem	2
4. Data Sources & Integration Strategy (4 Sources)	3
5. Microsoft Fabric & Ecosystem Tool Stack	4
6. Architecture & Implementation Plan	5
7. Expected Outcomes & Portfolio Impact.....	7

1. Introduction

In modern e-commerce, taking an order online is only half the job. Delivering that package to a customer's doorstep on time requires seamless coordination between internal inventory systems, external delivery carriers (like NZ Post or Aramex), third-party fulfillment invoices, and real-time road conditions.

This project designs an end-to-end data integration pipeline using **Microsoft Fabric** and **dbt**. It combines internal sales data, multi-cloud carrier logs, third-party invoice archives, and live New Zealand road alert data into a single, automated reporting system.

2. Aim & Purpose

- **Primary Aim:** Build a production-ready, 4-source data pipeline in Microsoft Fabric that automatically detects delivery delays, reconciles shipping costs, and audits carrier SLAs.
- **Technical Purpose:** Demonstrate complete mastery across the Microsoft Fabric platform-including Data Factory pipelines, PySpark notebooks, OneLake Shortcuts (AWS S3 & Azure Blob), T-SQL Data Warehousing with Stored Procedures, dbt modeling, security policies, and Direct Lake Power BI reporting.
- **Career Alignment:** Provide a concrete, real-world project for your portfolio while directly preparing you for the **DP-700 / DP-600** certification exams.

3. Real-World Business Problem

When a customer orders an item from an online retailer in New Zealand, four major operational challenges occur:

- **Multi-Cloud Data Disconnect:** Order data lives in an internal SQL database, carrier tracking logs arrive in AWS S3, and billing/rate cards sit in Azure Blob Storage. The business lacks a unified view across these environments.
- **Unnoticed Delays:** Severe weather or road closures on State Highways disrupt transit routes, but the business only learns about delays after customers lodge complaints.
- **Unvalidated Carrier Invoices:** Shipping partners bill the retailer monthly, but the retailer has no automated way to cross-reference invoiced rates against actual package delivery timestamps.

- **Lack of Data Governance & Lineage:** Business teams struggle to trust metrics because transformations are hidden in ad-hoc scripts without documentation or testing.

This project solves these issues by creating a single, enterprise-grade data hub.

4. Data Sources & Integration Strategy (4 Sources)

To prove you can handle diverse enterprise data environments, this pipeline ingests from four distinct sources:

Source #	Source Name	What Data It Holds	Format	Ingestion / Integration Method
1	Operational E-Commerce DB	Customers, Orders, Products, Fulfillment Depots (Auckland & Christchurch)	Relational SQL (T-SQL)	Data Factory Copy Activity (Dynamic watermarking)
2	Live NZ Transport API	Real-time road closures, weather hazards, state highway incidents	REST API (JSON)	Fabric Python Notebook (requests + pydantic schema validation)
3	AWS S3 Cloud Storage	Carrier scan logs (pickup, depot transfer, delivery scans)	Parquet / CSV	OneLake S3 Shortcut (Zero-copy file virtualization)
4	Azure Blob Storage	Historical carrier rate cards and third-party fulfillment invoices	Parquet / Delta	OneLake Azure Blob Shortcut (Zero-copy file virtualization)

5. Microsoft Fabric & Ecosystem Tool Stack

This project leverages every core component of Microsoft Fabric alongside modern transformation tools:

- **Fabric Data Factory:** Orchestrates master execution flow, schedules jobs, triggers notebooks, and manages error loops.
- **OneLake & Lakehouse:** Stores raw files (**Bronze Layer**) and cleaned Delta tables (**Silver Layer**). Uses **V-Order** for rapid read execution.
- **OneLake Shortcuts:** Connects external **AWS S3** and **Azure Blob Storage** buckets directly into OneLake without running data copy pipelines.
- **Fabric Notebooks (PySpark & Python):** Ingests live API payloads, executes Pydantic schema enforcement, catches corrupted records, and writes to Error_Log tables.
- **Fabric Data Warehouse:** Hosts the structured **Gold Layer** analytical tables using T-SQL engines.
- **dbt (dbt-fabric / dbt-sqlserver):** Transforms Silver Delta tables into Gold Star Schema models (Fact Orders, Dim Customer, Dim Carrier, Dim Depot), runs data quality assertions (dbt test), and generates interactive lineage documentation.
- **T-SQL Stored Procedures:** Manages complex warehouse tasks, dynamic MERGE logic (SCD Type 1 & Type 2), audit log entries, and automated batch controls.
- **Governance & Security:** Applies **Row-Level Security (RLS)** in T-SQL so regional depot managers access only their assigned territory data.
- **Power BI (Direct Lake Mode):** Renders real-time delivery performance dashboards directly off Gold Delta tables with **zero import latency**.

6. Architecture & Implementation Plan

The project applies the **Medallion Architecture** (Bronze → Silver → Gold), splitting duties logically across PySpark, dbt, and T-SQL:

BRONZE LAYER

- T-SQL (Data Factory Copy)
- Live NZTA API (Python Notebook)
- AWS S3 Logs (OneLake Shortcut)
- Azure Blob Invoices (OneLake Shortcut)

SILVER LAYER

- Fabric Lakehouse Delta Tables
- PySpark cleaning, JSON flattening, & Pydantic schema validation
- Dead-letter error queue for bad API payloads

GOLD LAYER

- Fabric Data Warehouse
- dbt modeling (Star Schema, dbt test assertions, lineage documentation)
- T-SQL Stored Procedures (Incremental MERGE, SCD Type 2 history tracking)
- Row-Level Security (RLS) policies for regional access control

DIRECT LAKE POWER BI REPORT

- Real-time delivery ETA tracking, carrier SLA auditing, & delay alerts

Step-by-Step Execution Plan

1. Ingestion & Zero-Copy Virtualization (Bronze):

- Configure Data Factory Copy Activity to extract T-SQL transactional tables.
- Create two **OneLake Shortcuts** pointing to AWS S3 (carrier scans) and Azure Blob Storage (invoices).
- Write a Python script (requests + pydantic) in a Fabric Notebook to fetch live NZTA road warnings.

2. Standardization & Quality Enforcement (Silver):

- Run PySpark notebooks to flatten nested API JSON and standardize timestamps into NZ local time.
- Isolate malformed or missing records into an Error_Log Delta table.
- Write clean records into Silver Delta tables with **V-Order** optimization enabled.

3. Data Modeling & Procedural Upserts (Gold):

- Connect **dbt** to the Fabric SQL Analytics Endpoint / Warehouse.
- Write dbt models to build a Gold Star Schema and execute dbt test to ensure primary key uniqueness and referential integrity.
- Write T-SQL **Stored Procedures** using MERGE statements to handle SCD Type 2 tracking (e.g., updating customer address history or depot manager assignments).

4. Security & Reporting:

- Configure T-SQL Row-Level Security (RLS) to restrict Auckland Depot managers to North Island records and Christchurch managers to South Island records.
- Build a **Direct Lake Power BI** dashboard showing real-time order status, weather delay warnings, and carrier invoice reconciliation.

7. Expected Outcomes & Portfolio Impact

Upon completion, this project delivers four high-impact portfolio assets:

- **Enterprise GitHub Repository:** Clean directory containing Python ingestion code, PySpark transformation scripts, dbt models/tests, T-SQL Stored Procedures, and Data Factory JSON configs.
- **dbt Docs & Lineage Graph:** Interactive documentation showing visual data lineage from Bronze Lakehouse sources to Gold Warehouse tables.
- **Architecture & Production Blueprint:** A clear story explaining how you solved a complex 4-source retail problem in New Zealand using modern data tools.
- **DP-700 / DP-600 Mastery:** Direct, practical experience covering key certification objectives—including OneLake Shortcuts, PySpark validation, dbt modeling, T-SQL Stored Procedures, Fabric Security (RLS), and Direct Lake mode.